



Glosario

M1. Fundamentos de programación



Glosario

» 1

Acoplamiento

Medida para establecer las cualidades que tiene un sistema, en relación al grado de dependencia que existe entre los diferentes componentes o módulos que lo integran. Indica cómo interactúan o dependen los módulos, unos de otros, para realizar tareas. Un buen diseño de software, busca minimizar el acoplamiento, de modo que los componentes estén suficientemente separados para ser independientes, pero aún así, cooperen de manera adecuada.

» 2

Alcance

Definición de las funciones, características y límites de un proyecto o sistema, especificando qué elementos se incluirán y cuáles quedarán excluidos.

» 3

Algoritmo

Es una secuencia de instrucciones finitas y claras diseñadas para resolver un problema. Debe ser eficiente, preciso, terminable y capaz de generar una salida correcta para las entradas dadas. La capacidad de un algoritmo para resolver problemas de manera efectiva lo convierte en un pilar fundamental de la programación y la informática.

» 4

Anidamiento

Se refiere a la práctica de incluir una estructura de control, dentro de otra.

› 5

Arquitectura

Estructura fundamental de un sistema, incluyendo sus componentes, las relaciones entre ellos y los principios que guían su diseño y evolución. Una arquitectura bien diseñada, facilita el mantenimiento, la escalabilidad y la calidad del sistema.

› 6

Calidad

Grado en que el software cumple con los requerimientos funcionales y no funcionales, tomando en consideración las expectativas de los usuarios finales.

› 7

Código "smelly"

Se refiere a fragmentos del código que, aunque funcionan correctamente, indican un mal diseño o prácticas de codificación deficientes. Estos fragmentos suelen ser difíciles de mantener o escalar, y son una señal de que el código requiere refactorización. Algunos ejemplos incluyen funciones demasiado largas, nombres poco descriptivos o excesivas dependencias.

› 8

Cohesión

Se refiere a la valoración que se hace, para determinar el grado en que las tareas realizadas por un componente están interconectadas y contribuyen a que se cumpla de forma eficiente, con el objetivo principal que se le asignó a dicho componente. Un buen diseño de software busca una alta cohesión.

» 9

Despliegue

Es el proceso de preparar, instalar y poner en funcionamiento un sistema de software en el entorno del usuario final. Incluye actividades como la instalación, configuración y pruebas, con el fin de asegurar que el sistema funcione correctamente en su entorno operativo.

» 10

Deuda técnica

Metáfora que representa cuál podría ser el costo acumulado de decisiones de diseño o desarrollo, cuando se opta por soluciones rápidas, pero que van a requerir de una refactorización o corrección en el futuro.

» 11

Documentación

Conjunto de materiales que describen el diseño, funcionamiento, uso y mantenimiento de un sistema o componente de software.

» 12

Escenario

Descripción narrativa de una secuencia de eventos que ilustran situaciones de uso o interacción con un sistema. Se utiliza para comprender y validar requisitos, proporcionando el contexto, sobre cómo los usuarios interactúan con el sistema en situaciones reales.

» 13

Estimación

Proceso de prever el esfuerzo, el tiempo y los recursos necesarios para completar un proyecto o tarea. Utiliza datos históricos y técnicas analíticas para proporcionar

proyecciones precisas que guíen la planificación y ejecución del proyecto.

» 14

Estructuras de control

Sirven para definir el flujo de ejecución de un programa, determinando qué partes del código se ejecutan, cuántas veces y bajo qué condiciones.

» 15

Estándar

Conjunto de criterios o especificaciones establecidos por una autoridad reconocida, para garantizar la calidad, compatibilidad e interoperabilidad de los productos o de los procesos.

» 16

Función

Bloque de código que realiza una tarea específica.

» 17

IDE

Entorno de Desarrollo Integrado o IDE (por sus siglas en inglés) es una herramienta de software que combina funciones esenciales para la programación en una sola interfaz, como un editor de texto, un compilador, un depurador y un gestor de proyectos, algunos ejemplos son: Visual Studio Code, IntelliJ IDEA y Eclipse.

» 18

Integración

Proceso de combinar componentes o módulos individuales, para formar un siste-

ma completo que funcione de manera cohesiva.

› 19

Interfaz de usuario

Medio a través del cual los usuarios interactúan con un sistema, abarcando elementos visuales, controles y mecanismos de navegación. Una interfaz efectiva combina estética, accesibilidad y funcionalidad, para mejorar la experiencia del usuario.

› 20

Lexis

Es el conjunto de caracteres permitidos en un lenguaje de programación para construir instrucciones, es decir, las letras, números y símbolos que se pueden usar. También existen palabras reservadas llamadas “token”, estas son las piezas más pequeñas de significado en un lenguaje de programación.

› 21

Marco de trabajo

También conocidos como frameworks, son estructuras de software reutilizable que proporcionan un conjunto de componentes y directrices, para desarrollar aplicaciones de manera eficiente. Por ejemplo, frameworks como Django para Python o Spring para Java, ofrecen soluciones listas para manejar tareas comunes, como la gestión de bases de datos o la creación de APIs.

› 22

Metodología

Conjunto de métodos, procedimientos y prácticas, utilizados en una disciplina para alcanzar objetivos específicos, especialmente en el desarrollo de software.

» 23

Necesidad

Es una función esencial que un sistema debe cumplir, para resolver un problema específico de un usuario. Identificar correctamente estas necesidades, es crucial para que el desarrollo del software resulte exitoso. De una necesidad se pueden derivar múltiples requerimientos.

» 24

Operación

En el contexto del software, operación se refiere a la etapa en la que un sistema está completamente desplegado y funcionando en su entorno productivo. Incluye actividades como monitoreo, mantenimiento y soporte para garantizar que el sistema funcione correctamente y cumpla con las expectativas del usuario.

» 25

Recursos

Elementos necesarios para llevar a cabo un proyecto, como: personal, hardware, recursos financieros, programas, entre otros.

» 26

Refactorizar

Proceso para reestructurar el código existente, sin cambiar su comportamiento externo. El objetivo es mejorar su diseño, facilitar el mantenimiento y optimizar el rendimiento. La refactorización corrige problemas como el "código smelly" y promueve buenas prácticas de programación.

» 27

Rendimiento

Mide la eficiencia con la que un programa o sistema realiza sus funciones, bajo condiciones específicas, como los tiempos de respuesta y el uso de recursos. Es crucial para garantizar que el sistema opere de manera efectiva y cumpla con las expectativas del usuario.

» 28

Requerimiento

Un requerimiento es una declaración clara y específica, que describe una característica o restricción, que un sistema debe cumplir para ser considerado exitoso. Los requerimientos son fundamentales en el desarrollo de software, ya que guían la planificación, el diseño, la implementación y las pruebas del sistema.

Estos se dividen en:

- **Requerimientos funcionales:** Detallan lo que el sistema debe hacer, como procesos o funciones específicas.
- **Requerimientos no funcionales:** Describen de qué manera debe operar el sistema, teniendo en cuenta aspectos como el rendimiento, la seguridad o la usabilidad.

» 29

Semántica

La semántica de un código representa su 'sentido' o la 'lógica' utilizada para resolver el problema. Se refiere a la manera como se ejecutan las instrucciones, y a los valores con los que se sustituyen las variables, durante la ejecución del código. Todo ello, con el fin de garantizar el buen funcionamiento de un programa.

» 30

Sintaxis

Es el conjunto de reglas, que determinan cómo se deben organizar los elementos, como las palabras y sus significados (el léxico), para escribir instrucciones que sean válidas y comprensibles por el lenguaje de programación.

» 31

Software

Es un conjunto de programas, datos y documentación asociados entre sí, que permiten a una computadora realizar tareas específicas. Incluye aplicaciones, sistemas operativos y herramientas que facilitan diversas funciones.

» 32

Tipo de datos

Clasificación que define qué tipo de valores puede almacenar una variable y qué operaciones pueden realizarse, utilizando esos valores.

» 33

Validación

Proceso de evaluar un sistema o componente, durante o al final de su desarrollo, para determinar si cumple los requerimientos especificados y las necesidades del cliente. Implica pruebas de aceptación y garantiza que el producto final cumple con las expectativas del usuario.

» 34

Variable

Un contenedor o espacio en la memoria física o virtual de una computadora, donde se almacenan distintos tipos de datos (valores) durante la ejecución de un programa.

› Referencias:

1. Sipser, M. (2012). Introduction to the Theory of Computation (3rd ed.). Cengage Learning.
2. Ferreira, W. (2017). Computer Science Distilled: Learn the Art of Solving Computational Problems. Code Energy.
3. Washizaki, H. (2024). Guide to the Software Engineering Body of Knowledge v4.0. IEEE Computer Society

Elaboró contenido:

Alan García y Marco Ruíz